

République du Sénégal



Un Peuple – Un But – Une Foi

Ministère de l'Enseignement Supérieur et de la Recherche

Université Alioune DIOP de Bambey



UFR Sciences Appliquées et Technologies de l'Information et de la
Communication (SATIC)

Département Technologies de l'Information et de la Communication
(TIC)

THEME

Sécurisation des Communications avec mTLS et SSL/TLS

Réalisé par :

- Ousmane Diallo

Encadré par :

- Dr. Maodo Diop

ANNEE ACADEMIQUE 2024-2025

Table des matières

Introduction.....	3
I. Mise en place du SSL/TLS sur le backend Spring Boot	3
I.1. Génération du certificat serveur	3
I.2. Configuration de Spring Boot pour activer HTTPS.....	4
Résultat.....	5
I.3. Test de la connexion HTTPS avec CURL	5
I.4. Accès HTTPS depuis Postman	6
II. Mise en place du mTLS (Mutual TLS) dans le backend Spring Boot.....	7
✓ Le keystore du serveur (server-keystore.p12)	7
✓ Le truststore du serveur (server-truststore.p12).....	7
II.1 Créer un truststore pour le serveur.....	7
II.1.1. Génération du certificat client.....	7
Objectif Principal : Authentification Mutuelle	7
Paramètres expliqués :	8
-genkeypair.....	8
-alias client-cert	8
-keyalg RSA	8
-keysize 2048	8
-validity 365	8
-keystore client-keystore.p12.....	8
-storetype PKCS12	8
II.1.2. Créer un truststore pour le serveur :	9
II.2. Configuration mTLS dans Spring Boot	9
II.3. Vérification et fonctionnement du mTLS	10
Échange mTLS :	10
Cas d'échec	10
II.3.1. Test du mTLS.....	10
III. Mise en place du SSL/TLS sur le front-end Angular	11
III.1. Problématique : Angular en local et certificats HTTPS.....	12
III.2. Génération du certificat client (front-end).....	12
III.3. Création du serveur proxy pour le (front-end).....	13
IV. Vulnérabilités OWASP liées aux certificats SSL/TLS et mTLS	14
Conclusion Générale.....	17

Introduction

Dans un contexte où les cyberattaques sont de plus en plus fréquentes, la sécurisation des communications entre les applications est devenue un enjeu majeur. Les échanges entre un client (navigateur, application mobile, front-end) et un serveur doivent garantir la confidentialité, l'intégrité des données ainsi que l'authentification des parties. C'est dans ce cadre que les protocoles **SSL/TLS (Secure Sockets Layer / Transport Layer Security)** et **mTLS (mutual TLS)** jouent un rôle essentiel.

SSL/TLS permet de chiffrer les données échangées et de vérifier l'identité du serveur, tandis que mTLS étend ce mécanisme en imposant également l'authentification du client grâce à un certificat numérique. Ce modèle renforce la sécurité et empêche tout accès non autorisé même en cas d'interception de requêtes.

Dans le cadre de ce projet, il s'agissait de comprendre et mettre en œuvre ces mécanismes de sécurité au travers :

- De la génération et l'utilisation de certificats publics/privés du client et du serveur ;
- De l'utilisation d'outils open source tels que keytool, OpenSSL, ou Let's Encrypt ;
- De tests via des outils comme curl en mode verbeux (curl -vk) ;
- Et de la consommation d'un ou plusieurs endpoints sécurisés.

Le projet impose également d'étudier les protocoles modernes de communication (REST, GraphQL, SOAP, gRPC), les principaux algorithmes cryptographiques utilisés aujourd'hui, ainsi que les vulnérabilités recensées par OWASP concernant la gestion des certificats.

L'implémentation a été réalisée avec Spring Boot pour une application de E-Commerce qu'on a mis en place et de produire une API sécurisée en HTTPS et protégée par mTLS. Une application cliente consomme ensuite ces endpoints en présentant son certificat, rendant la communication entièrement sécurisée des deux côtés.

I. Mise en place du SSL/TLS sur le backend Spring Boot

Dans cette section, nous présentons les différentes étapes permettant de sécuriser notre application backend Spring Boot grâce au protocole **SSL/TLS**. Cette configuration assure le chiffrement des communications entre le client (navigateur, Postman, application Angular/React, etc.) et le serveur. Elle constitue la base nécessaire avant d'activer le mécanisme complémentaire du **mTLS** (authentification mutuelle par certificat).

I.1. Génération du certificat serveur

Pour activer SSL/TLS dans Spring Boot, il est d'abord nécessaire de générer un certificat **X.509** au format **PKCS12**.

Ce certificat contient :

- Une **clé privée** (priv.key)
- Une **clé publique**
- Les métadonnées (CN, organisation, pays...)
- Le certificat signé (auto-signé dans notre cas)

Nous avons utilisé **keytool**, l'outil fourni avec le JDK :

```
Windows PowerShell
PS C:\Users\DELL> keytool -genkeypair -alias server-cert -keyalg RSA -keysize 2048 -validity 365 -keystore server-keystore.p12 -storetype PKCS12
Enter keystore password:
Keystore password is too short - must be at least 6 characters
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
What is your first and last name?
What is the name of your organizational unit?
What is the name of your organization?
What is the name of your City or Locality?
What is the name of your State or Province?
What is the two-letter country code for this unit?
Is CN=ousmane diallo, OU=Unknown, O=Unknown, L=bambey, ST=bambey, C=sn correct?
[no]: yes
Generating 2048-bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 365 days
for: CN=ousmane diallo, OU=Unknown, O=Unknown, L=bambey, ST=bambey, C=sn
PS C:\Users\DELL>
```

Paramètres importants

- **alias** : nom du certificat
- **RSA 2048 bits** : algorithme cryptographique recommandé
- **PKCS12** : format standard utilisé par Java
- **validity** : durée de validité du certificat
- **server-keystore.p12** : fichier contenant la clé privée + certificat

	static	21/08/2025 21:52	Dossier de fichiers	
	templates	21/08/2025 21:52	Dossier de fichiers	
	application	06/09/2025 21:52	Fichier source Proper...	1 Ko
	server-keystore	14/11/2025 12:26	Échange d'informatio...	3 Ko

Ce certificat sera utilisé par Spring Boot pour activer la couche HTTPS sur le port **8443**.

I.2. Configuration de Spring Boot pour activer HTTPS

Dans le fichier **application.properties**, nous avons activé SSL/TLS :

```
24
25 # SSL Configuration
26 server.port=8443
27 server.ssl.enabled=true
28 server.ssl.key-store=classpath:server-keystore.p12
29 server.ssl.key-store-password=changeit
30 server.ssl.key-store-type=PKCS12
31
32
```

Résultat

- Le serveur Spring Boot n'accepte plus les requêtes HTTP sur le port 8080.
- Toutes les communications doivent maintenant passer par [HTTPS://localhost:8443](https://localhost:8443).
- Le serveur présente automatiquement son certificat au client.

I.3. Test de la connexion HTTPS avec CURL

Pour vérifier que SSL/TLS est bien actif :

`curl -vk https://localhost:8443/api/produits` pour lister quelques produits enregistrer

Explications :

- `-k` permet d'ignorer le fait que le certificat est auto-signé
- `-v` affiche les détails de la négociation SSL/TLS

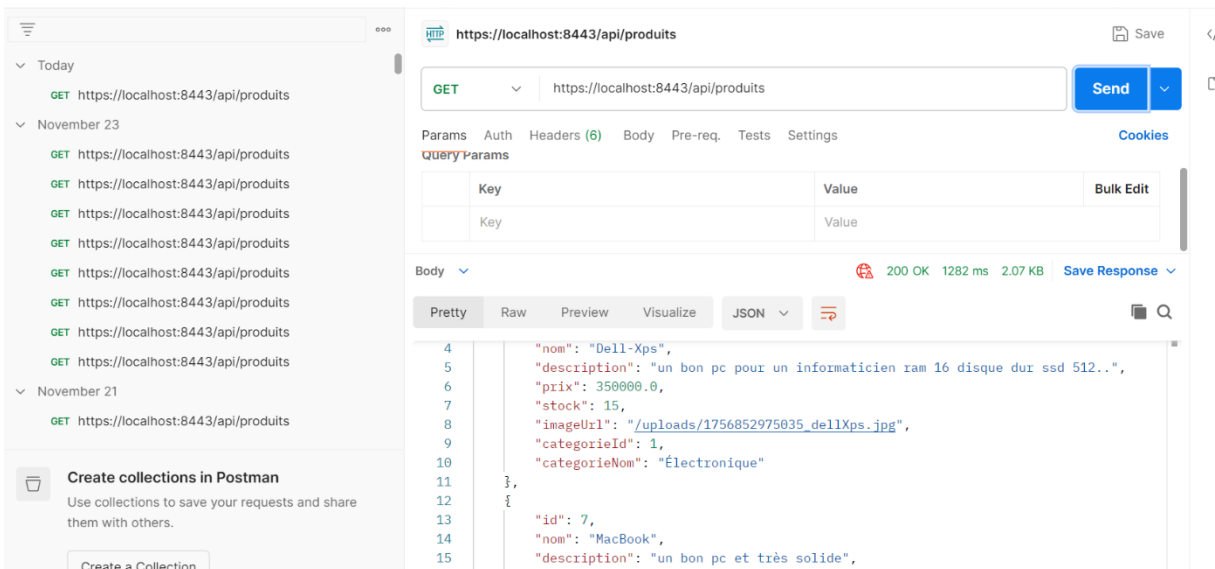
```
PS C:\Users\DELL> curl.exe -vk https://localhost:8443
* Host localhost:8443 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8443...
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* ALPN: server did not agree on a protocol. Uses default.
* Established connection to localhost (::1 port 8443) from ::1 port 50580
* using HTTP/1.x
> GET / HTTP/1.1
> Host: localhost:8443
> User-Agent: curl/8.16.0
> Accept: */*
>
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
* Request completely sent off
< HTTP/1.1 403
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Set-Cookie: JSESSIONID=8F83A92AB510D7A63EBD581C2DAB6750; Path=/; Secure; HttpOnly
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 0
< Cache-Control: no-cache, no-store, max-age=0, must-revalidate
< Pragma: no-cache
< Expires: 0
< Strict-Transport-Security: max-age=31536000 ; includeSubDomains
< Set-Cookie: JSESSIONID=8F83A92AB510D7A63EBD581C2DAB6750; Path=/; Secure; HttpOnly
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 0
< Cache-Control: no-cache, no-store, max-age=0, must-revalidate
< Pragma: no-cache
< Expires: 0
< Strict-Transport-Security: max-age=31536000 ; includeSubDomains
< X-Frame-Options: DENY
< Content-Length: 0
< Date: Fri, 14 Nov 2025 15:14:09 GMT
<
* Connection #0 to host localhost:8443 left intact
PS C:\Users\DELL> |
```

```

PS C:\Users\DELL> curl.exe -vk https://localhost:8443/api/produits
* Host localhost:8443 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8443...
* schannel: disabled automatic use of client certificate
* ALPN: curl offers http/1.1
* ALPN: server did not agree on a protocol. Uses default.
* Established connection to localhost (::1 port 8443) from ::1 port 59482
* using HTTP/1.x
> GET /api/produits HTTP/1.1
> Host: localhost:8443
> User-Agent: curl/8.16.0
> Accept: */*
>
* schannel: remote party requests renegotiation
* schannel: renegotiating SSL/TLS connection
* schannel: SSL/TLS connection renegotiated
* Request completely sent off
< HTTP/1.1 200
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 0
< Cache-Control: no-cache, no-store, max-age=0, must-revalidate
< Pragma: no-cache
< Expires: 0
< Strict-Transport-Security: max-age=31536000 ; includeSubDomains
< X-Frame-Options: DENY
< X-Frame-Options: DENY
< Content-Type: application/json
< Transfer-Encoding: chunked
< Date: Fri, 14 Nov 2025 15:49:48 GMT
<
[{"id":6,"nom":"Dell-Xps","description":"un bon pc pour un informaticien ram 16 disque dur ssd 512..","prix":350000.0,"stock":16,"imageUrl":"/uploads/1756852975035_dellXps.jpg","categorieId":1,"categorieNom":"Électronique"},{"id":7,"nom":"MacBook","description":"un bon pc et très solide","prix":360000.0,"stock":17,"imageUrl":"/uploads/1756855116016_Mac pro.jpg","categorieId":1,"categorieNom":"Électronique"},{"id":8,"nom":"Dell-XPS ","description":"i5 10th ram 16 disque dur ssd 256","prix":250000.0,"stock":14,"imageUrl":"/uploads/175685502953_dellxps13-9.webp","categorieId":1,"categorieNom":"Électronique"},{"id":9,"nom":"Dell-Xps 14","description":"i5 8th ram 8g ssd disk 256","prix":225000.0,"stock":24,"imageUrl":"/uploads/1756855754711_dellxps13.jpg","categorieId":1,"categorieNom":"Électronique"},{"id":10,"nom":"airpod pro2","description":"un bon écouteur de qualité ","prix":6000.0,"stock":99,"imageUrl":"/uploads/1756856297638_airpodpro2.webp","categorieId":1,"categorieNom":"Électronique"},{"id":11,"nom":"Hp Elite","description":"i5 8th ram 16go ssd 256","prix":160000.0,"stock":16,"imageUrl":"/uploads/1756856420336_HP EliteBook 840 G7 - open.jpg","categorieId":1,"categorieNom":"Électronique"},{"id":12,"nom":"iphone-xr ","description":"stockage 128go 12000f et 64 110000f","prix":120000.0,"stock":100,"imageUrl":"/uploads/1756857020508_iphone-xr-64gb.jpg","categorieId":1,"categorieNom":"Électronique"},{"id":13,"nom":"iphone 16","description":"stockage 256go et moins ","prix":550000.0,"stock":17,"imageUrl":"/uploads/1756857147198_iphone16.jpeg","categorieId":1,"categorieNom":"Électronique"}]* Connection #0 to host localhost:8443 left intact
PS C:\Users\DELL> |

```

I.4. Accès HTTPS depuis Postman



The screenshot shows the Postman application interface. On the left, there's a sidebar with a 'Today' section containing a single request to 'https://localhost:8443/api/produits'. Below it, there's a 'November 23' section with multiple requests to the same URL. At the bottom of the sidebar, there's a 'Create collections in Postman' section with a 'Create a Collection' button.

The main area of the interface shows a GET request to 'https://localhost:8443/api/produits'. The 'Send' button is visible. Below the URL bar, there are tabs for 'Params', 'Auth', 'Headers (6)', 'Body', 'Pre-req.', 'Tests', and 'Settings'. The 'Params' tab is selected, showing a table with 'Key' and 'Value' columns. Below this, the 'Body' tab is selected, showing the response body in JSON format. The response is a list of products with details like id, nom, description, prix, stock, imageUrl, categorieId, and categorieNom.

```

{
  "id": 6,
  "nom": "Dell-Xps",
  "description": "un bon pc pour un informaticien ram 16 disque dur ssd 512..",
  "prix": 350000.0,
  "stock": 16,
  "imageUrl": "/uploads/1756852975035_dellXps.jpg",
  "categorieId": 1,
  "categorieNom": "Électronique"
},
{
  "id": 7,
  "nom": "MacBook",
  "description": "un bon pc et très solide",
  "prix": 360000.0,
  "stock": 17,
  "imageUrl": "/uploads/1756855116016_Mac pro.jpg",
  "categorieId": 1,
  "categorieNom": "Électronique"
},
{
  "id": 8,
  "nom": "Dell-XPS ",
  "description": "i5 10th ram 16 disque dur ssd 256",
  "prix": 250000.0,
  "stock": 14,
  "imageUrl": "/uploads/175685502953_dellxps13-9.webp",
  "categorieId": 1,
  "categorieNom": "Électronique"
},
{
  "id": 9,
  "nom": "Dell-Xps 14",
  "description": "i5 8th ram 8g ssd disk 256",
  "prix": 225000.0,
  "stock": 24,
  "imageUrl": "/uploads/1756855754711_dellxps13.jpg",
  "categorieId": 1,
  "categorieNom": "Électronique"
},
{
  "id": 10,
  "nom": "airpod pro2",
  "description": "un bon écouteur de qualité ",
  "prix": 6000.0,
  "stock": 99,
  "imageUrl": "/uploads/1756856297638_airpodpro2.webp",
  "categorieId": 1,
  "categorieNom": "Électronique"
},
{
  "id": 11,
  "nom": "Hp Elite",
  "description": "i5 8th ram 16go ssd 256",
  "prix": 160000.0,
  "stock": 16,
  "imageUrl": "/uploads/1756856420336_HP EliteBook 840 G7 - open.jpg",
  "categorieId": 1,
  "categorieNom": "Électronique"
},
{
  "id": 12,
  "nom": "iphone-xr ",
  "description": "stockage 128go 12000f et 64 110000f",
  "prix": 120000.0,
  "stock": 100,
  "imageUrl": "/uploads/1756857020508_iphone-xr-64gb.jpg",
  "categorieId": 1,
  "categorieNom": "Électronique"
},
{
  "id": 13,
  "nom": "iphone 16",
  "description": "stockage 256go et moins ",
  "prix": 550000.0,
  "stock": 17,
  "imageUrl": "/uploads/1756857147198_iphone16.jpeg",
  "categorieId": 1,
  "categorieNom": "Électronique"
}
]

```

II. Mise en place du mTLS (Mutual TLS) dans le backend Spring Boot

Après avoir configuré le SSL/TLS unidirectionnel, la seconde étape a consisté à activer le **mTLS (Mutual TLS Authentication)**.

Contrairement au SSL classique où **seul le serveur présente un certificat**, le mTLS impose que **le client et le serveur s'authentifient mutuellement** via des certificats X.509. Ce mécanisme renforce considérablement la sécurité en empêchant qu'un utilisateur non autorisé puisse accéder aux API, même s'il contourne l'interface graphique.

Afin de permettre au serveur de vérifier les certificats clients, il a fallu préparer :

✓ Le **keystore** du serveur ([server-keystore.p12](#))

Contient :

- la clé privée du serveur
- le certificat public du serveur

(Composé avec `keytool`.)

✓ Le **truststore** du serveur ([server-truststore.p12](#))

Il permet au serveur d'identifier les certificats clients autorisés.

On y importe :

- le certificat **public** du client uniquement.

Cela permet à Spring Boot de valider qu'un client ayant la bonne identité tente bien d'accéder à l'API.

II.1 Créer un truststore pour le serveur

Le serveur doit *faire confiance* au certificat du client :

Pour réaliser **un truststore pour le serveur** Tu dois d'abord disposer du certificat public du client, par exemple essayons de créer un certificat pour un client :

II.1.1. Génération du certificat client

L'objectif principal de ce certificat client est **l'authentification du client** dans des communications sécurisées. Voici les objectifs détaillés

Objectif Principal : Authentification Mutuelle

- **Sans certificat** : Seul le serveur prouve son identité (HTTPS standard)
- **Avec certificat client** : Le client prouve aussi son identité au serveur
- **Résultat** : Authentification bidirectionnelle

```
Windows PowerShell
PS C:\Users\DELL> keytool -genkeypair -alias client-cert -keyalg RSA -keysize 2048 -validity 365 -keystore client-keystore.p12 -storetype PKCS12 -storepass changeit -keypass changeit
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
What is your first and last name?
[Unknown]: ousmane diallo
What is the name of your organizational unit?
[Unknown]:
What is the name of your organization?
[Unknown]:
What is the name of your City or Locality?
[Unknown]: bambey
What is the name of your State or Province?
[Unknown]: bambey
What is the two-letter country code for this unit?
[Unknown]: sn
Is CN=ousmane diallo, OU=Unknown, O=Unknown, L=bambey, ST=bambey, C=sn correct?
[no]: yes

Generating 2048-bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 365 days
for: CN=ousmane diallo, OU=Unknown, O=Unknown, L=bambey, ST=bambey, C=sn
PS C:\Users\DELL>
```

Paramètres expliqués :

-genkeypair

- **Fonction** : Génère une paire de clés (clé privée + certificat public)
- **Alternative** : -genkey (déprécié, même fonction)

-alias client-cert

- **Fonction** : Donne un nom identifiable à l'entrée dans le keystore
- **Usage** : Permet de référencer ce certificat plus tard avec ce nom

-keyalg RSA

- **Fonction** : Spécifie l'algorithme de chiffrement
- **Valeur** : RSA (Rivest-Shamir-Adleman) - algorithme asymétrique courant

-keysize 2048

- **Fonction** : Définit la longueur de la clé
- **Sécurité** : 2048 bits est considéré comme sécurisé actuellement

-validity 365

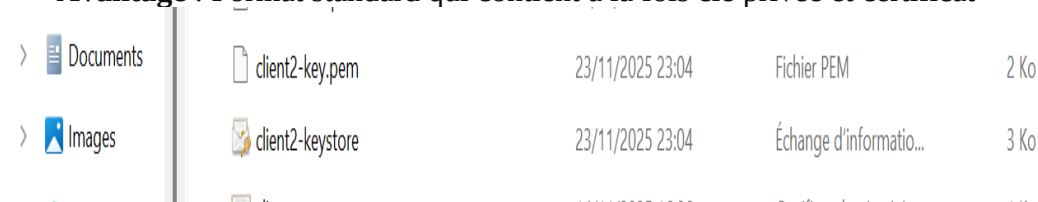
- **Fonction** : Durée de validité du certificat en jours
- **Valeur** : 365 jours (1 an)

-keystore client-keystore.p12

- **Fonction** : Nom du fichier qui contiendra la clé et le certificat
- **Format** : Fichier binaire sécurisé

-storetype PKCS12

- **Fonction** : Format du keystore
- **Avantage** : Format standard qui contient à la fois clé privée et certificat



> Documents	client2-key.pem	23/11/2025 23:04	Fichier PEM	2 Ko
> Images	client2-keystore	23/11/2025 23:04	Échange d'informatio...	3 Ko

Puis exporte le certificat public du client :
Par ce que

- **Keystore client** : Contient la clé privée (secrète) + certificat public
- **Fichier .cer** : Contient seulement le certificat public (peut être partagé)

```
PS C:\Users\DELL> keytool -export -alias client-cert -file client-cert.cer -keystore client-keystore.p12
Enter keystore password:
Certificate stored in file <client-cert.cer>
PS C:\Users\DELL>
```

II.1.2. Créer un truststore pour le serveur :

Le serveur doit *faire confiance* au certificat du client :

```
PS C:\Users\DELL> keytool -import -alias client-cert -file client-cert.cer -keystore server-truststore.p12 -storetype PKCS12
Enter keystore password:
Re-enter new password:
Owner: CN=ousmane diallo, OU=Unknown, O=Unknown, L=bambey, ST=bambey, C=sn
Issuer: CN=ousmane diallo, OU=Unknown, O=Unknown, L=bambey, ST=bambey, C=sn
Serial number: 385237905f9c0163
Valid from: Fri Nov 14 16:04:00 GMT 2025 until: Sat Nov 14 16:04:00 GMT 2026
Certificate fingerprints:
    SHA1: A4:95:9B:3E:E5:9A:FD:FC:9B:69:02:A3:FB:E9:56:39:47:C6:F0:9F
    SHA256: 3A:A5:46:B5:AE:57:B0:7C:2B:1D:71:6B:81:55:B7:FD:D3:07:E9:3D:3D:47:E5:50:5F:5C:28:3D:B0:9C:3D:1B
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:
#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 3C 3B 2F 2D 71 1B 3B BD 6C 1D 9E DB FC 5D 29 F2 <:/-q.;.l....)].
0010: 95 5F 9C 05
]
]

Certificate was added to keystore
PS C:\Users\DELL>
```

ues

 server-truststore

14/11/2025 16:14

Échange d'informatio...

2 Ko

II.2. Configuration mTLS dans Spring Boot

Dans le fichier application.properties, on ajoute les directives suivantes :

```
31
32 # mTLS - Authentification mutuelle
33 server.ssl.client-auth=need
34 server.ssl.trust-store=classpath:server-truststore.p12
35 server.ssl.trust-store-password=changeit
36 server.ssl.trust-store-type=PKCS12
37
```

La propriété **server.ssl.client-auth=need** impose que le client **doit obligatoirement** fournir un certificat lors de l'accès à l'API.

II.3. Vérification et fonctionnement du mTLS

Lorsque le client tente d'accéder à **https://localhost:8443/api/produits** :

Échange mTLS :

1. **Le serveur présente son certificat** → le client vérifie l'identité du serveur.
2. **Le client présente à son tour son certificat** → le serveur valide le certificat dans son truststore.
3. Si les deux identités sont valides, alors :
 - la connexion TLS est établie,
 - l'échange HTTP peut commencer.

Cas d'échec

Si le client :

- ne présente **aucun certificat**,
- présente un certificat **non reconnu**,
- ou présente un certificat **non signé par l'autorité attendue**,

→ Spring Boot renvoie **403 - Forbidden** ou une erreur TLS.

II.3.1. Test du mTLS

Deux méthodes ont été utilisées pour vérifier le bon fonctionnement :

1. Test avec cURL (ligne de commande)

```
curl.exe -vk https://localhost:8443/api/produits --cert client-keystore.p12:changeit --cert-type P12
```

CURL envoie :

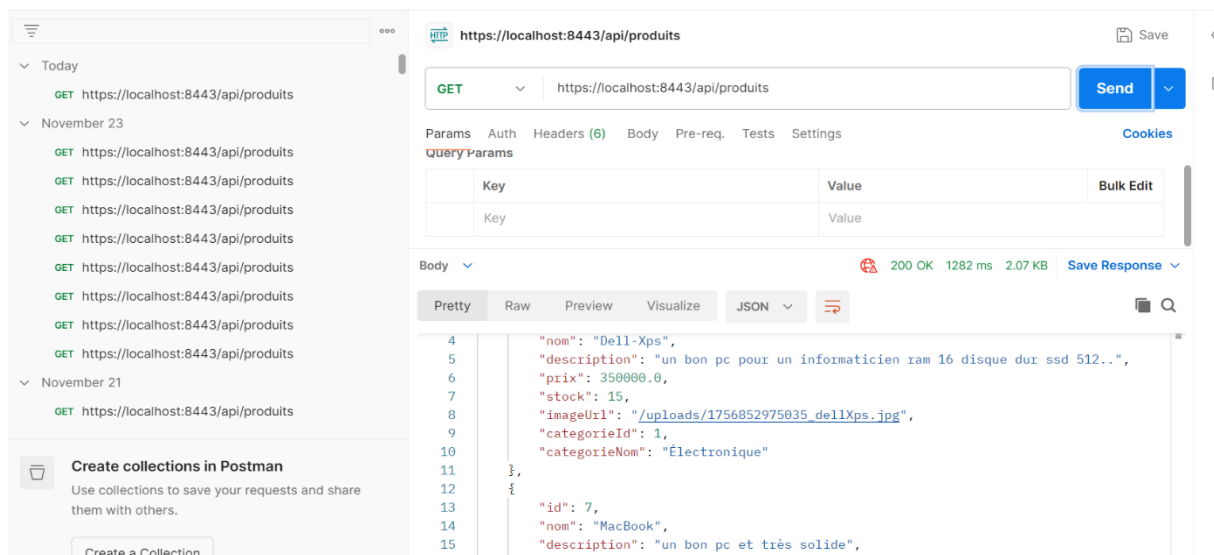
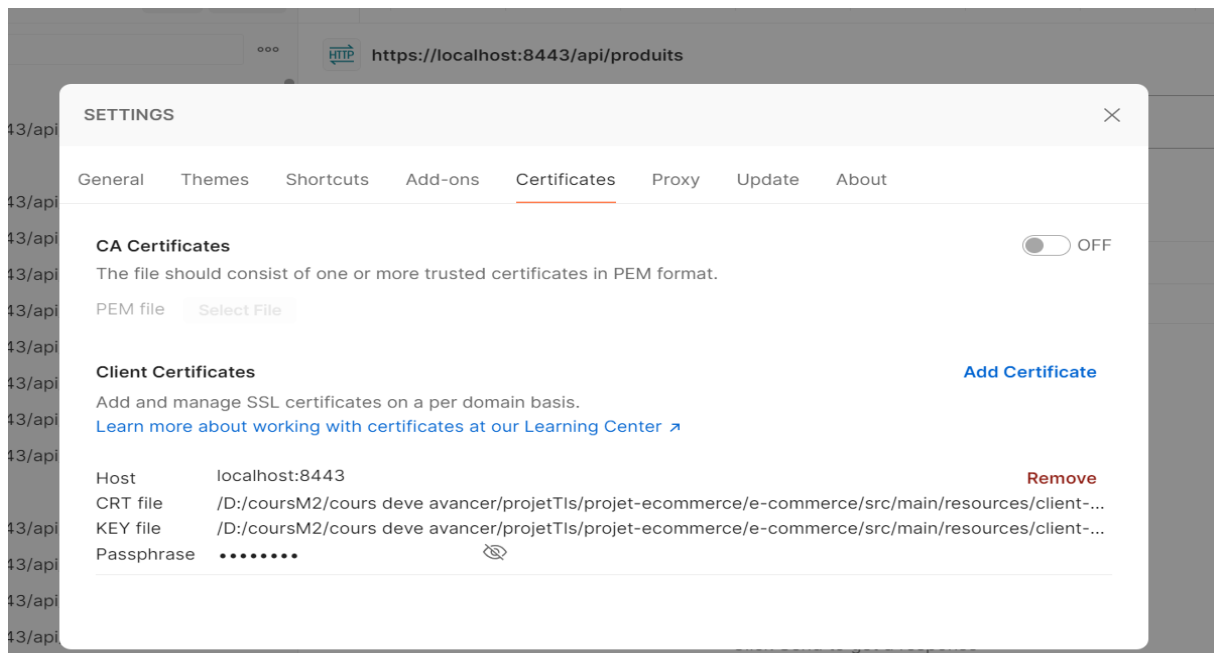
- la clé privée du client,
- le certificat du client,
- et initie l'authentification TLS mutuelle.

Un client sans certificat n'obtient pas de réponse valide.

2. Test avec Postman

Dans les paramètres SSL :

- Ajout du certificat client (PEM/P12)
- Mot de passe : changeit
- Ajout du certificat CA éventuellement



III. Mise en place du SSL/TLS sur le front-end Angular

Dans un scénario idéal, le front-end Angular peut communiquer directement en HTTPS avec le serveur Spring Boot. Toutefois, Angular en développement (port 4200) n'est **pas capable d'utiliser des certificats clients mTLS**, car :

- Le navigateur gère lui-même les certificats clients.
- Angular CLI ne permet pas nativement de configurer un keystore client pour mTLS.
- Les restrictions de sécurité du navigateur empêchent l'accès direct aux clés privées.

Pour contourner cette limitation, une solution consiste à **intercaler un proxy Node.js** entre Angular et le backend Spring Boot.

Le rôle du proxy est de :

[illegible]

III.3. Création du serveur proxy pour le (front-end)

```
D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front_Ecommerce>mkdir proxy-server

D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front_Ecommerce>cd proxy-server

D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front_Ecommerce\proxy-server>npm init -y
Wrote to D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front_Ecommerce\proxy-server\package.json:

{
  "name": "proxy-server",
  "version": "1.0.0",
  "description": "",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front_Ecommerce\proxy-server>
```

ensuite :

```
D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front_Ecommerce\proxy-server>npm install express http-proxy-middleware

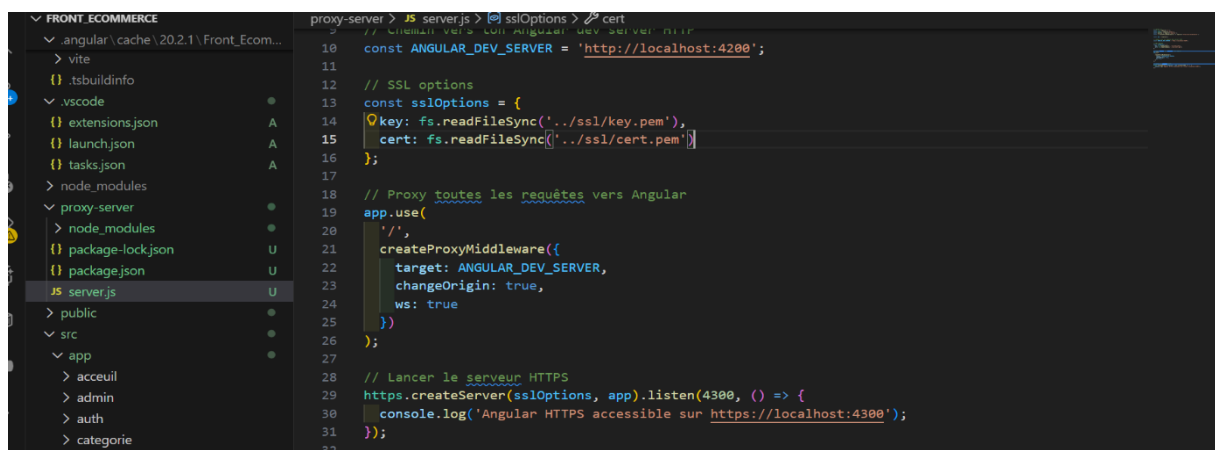
added 83 packages, and audited 84 packages in 41s

20 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front_Ecommerce\proxy-server>
```

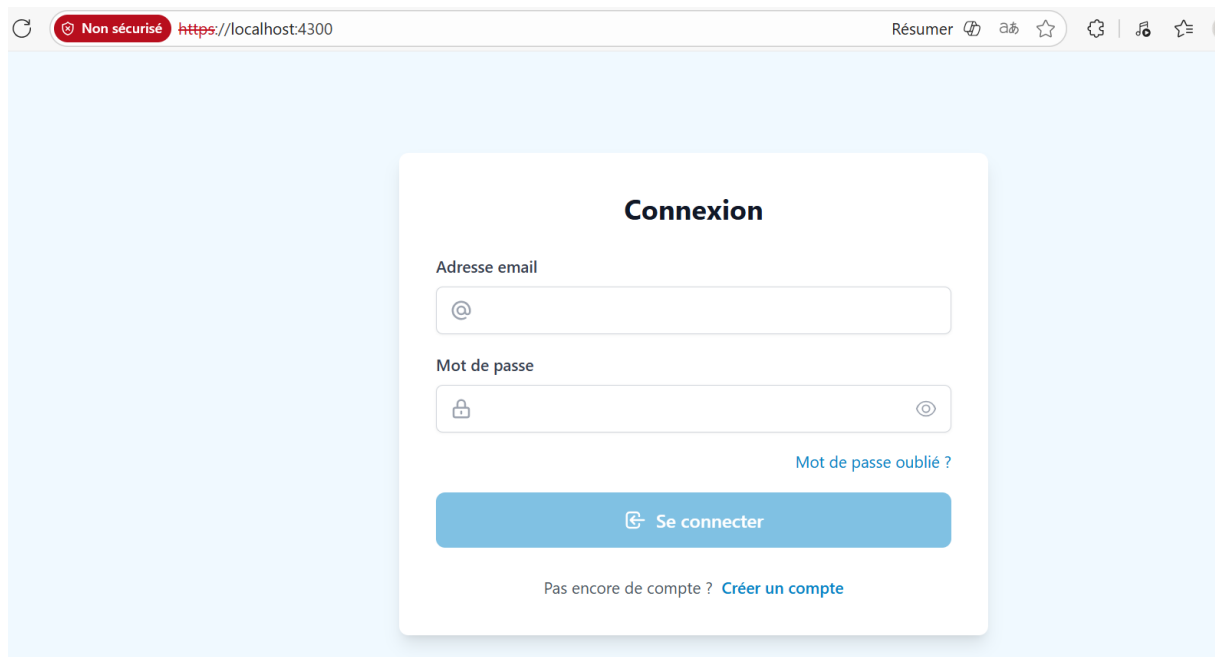
Nous avons créé un fichier proxy-server.js contenant la configuration HTTPS + certificat client :



```
10 const ANGULAR_DEV_SERVER = 'http://localhost:4200';
11
12 // SSL options
13 const sslOptions = {
14   key: fs.readFileSync('../ssl/key.pem'),
15   cert: fs.readFileSync('../ssl/cert.pem')
16 };
17
18 // Proxy toutes les requêtes vers Angular
19 app.use(
20   '/',
21   createProxyMiddleware({
22     target: ANGULAR_DEV_SERVER,
23     changeOrigin: true,
24     ws: true
25   })
26 );
27
28 // Lancer le serveur HTTPS
29 https.createServer(sslOptions, app).listen(4300, () => {
30   console.log('Angular HTTPS accessible sur https://localhost:4300');
31 });
32
```

	Nom	Modifié le	Type	Taille
> Documents	cert.pem	22/11/2025 16:02	Fichier PEM	3 Ko
> Images	key.pem	22/11/2025 15:58	Fichier PEM	4 Ko
> ousmane : per				
> Musique				
> Téléchargemen				

Maintenant pour lancer le serveur angular en https on lance d'abord le serveur angular avec **ng serve** ensuite on lance le serveur **node js** maintenant toutes les requêtes vont passer sur <https://localhost:4300>.



IV. Vulnérabilités OWASP liées aux certificats SSL/TLS et mTLS

La sécurité basée sur les certificats (SSL/TLS et mTLS) est robuste, mais plusieurs vulnérabilités sont répertoriées par l'OWASP. Ces failles apparaissent lorsqu'un système est mal configuré, utilise des certificats faibles ou ne respecte pas les bonnes pratiques cryptographiques.

Voici les principales vulnérabilités OWASP à connaître :

Certificats auto-signés non validés

Utiliser un certificat auto-signé **sans validation** peut permettre à un attaquant d'intercepter le trafic.

Risque : attaques Man-in-the-Middle (MITM).

Solution : utiliser un certificat émis par une CA (Let's Encrypt, DigiCert...), ou importer manuellement le certificat dans le truststore.

Utilisation d'algorithmes faibles

Certains algorithmes cryptographiques sont considérés obsolètes :

- MD5, SHA1 pour les signatures
- RSA < 2048 bits
- DH/ECDH faibles

Risque : falsification de certificat, déchiffrement du trafic.

Solution : utiliser RSA ≥ 2048 bits ou ECDSA, SHA-256 ou supérieur.

Mauvaise configuration SSL/TLS

Très fréquent selon OWASP.

Exemples :

- support de TLS 1.0 / 1.1 (deprecated)
- suites cryptographiques faibles
- compression TLS activée (attaque CRIME)
- mauvais ordre de préférence des ciphers

Risque : compromission de la confidentialité.

Solution : forcer TLS 1.2 / 1.3, désactiver les ciphers faibles.

Absence de validation du certificat côté client

Certains clients n'appliquent pas correctement les validations :

- expiration non vérifiée
- CN ou SAN non vérifiés
- chaîne de certification non validée

Risque : usurpation de serveur.

Solution : toujours vérifier :

la chaîne complète

la date d'expiration

le CN / SAN correspondant au domaine

Certificat expiré ou proche d'expiration

Une des erreurs les plus courantes.

Risque : indisponibilité du service, rejet automatique par les clients.

Solution : automatiser le renouvellement (Let's Encrypt Certbot, scripts).

Stockage des clés privées non sécurisé

Une clé privée compromise = identité du serveur ou du client compromise.

Risque : MITM, impersonation totale.

Solution :

- protéger les fichiers `.key` / keystore par mot de passe
- restreindre les permissions disque
- ne jamais versionner les clés privées (GitHub, GitLab)

Credential stuffing ou vol de certificats mTLS

Dans mTLS, le certificat client est une forme d'authentification.
Si un attaquant obtient un certificat compromis :

Il peut agir comme un client légitime.

Risque : accès complet aux API privées.
Solution :

- révoquer les certificats compromis
- rotation régulière
- utiliser OCSP/CRL

Absence de révocation de certificat

Si un certificat est volé mais **non révoqué**, il reste utilisable.

Risque : compromission prolongée.
Solution : utiliser :

- OCSP (Online Certificate Status Protocol)
- CRL (Certificate Revocation List)

Mauvaise utilisation des proxy ou du TLS termination

Sur certains serveurs, un proxy (Nginx, Node proxy, HAProxy...) termine la connexion TLS, mais le backend continue en HTTP simple.

Risque : exposition interne + MITM interne.
Solution : sécuriser les communications internes ou activer TLS end-to-end.

Conclusion Générale

La mise en place du SSL/TLS et du mTLS dans un environnement applicatif moderne constitue une étape essentielle pour garantir la confidentialité, l'intégrité et l'authenticité des échanges entre un client et un serveur. Dans ce projet, nous avons pu expérimenter concrètement ces mécanismes à travers la génération, la gestion et l'utilisation de certificats numériques, ainsi que l'intégration de ces certificats au sein d'une application complète composée d'un backend Spring Boot et d'un frontend Angular adossé à un proxy Node.js.

Ensuite, nous avons mis en œuvre le **mutual TLS (mTLS)**, un niveau plus avancé de sécurité où le serveur authentifie également le client grâce à un certificat client. Cette double authentification renforce fortement la sécurité du système, notamment dans les contextes d'APIs sensibles, d'intégrations inter-services ou d'architectures Zéro Trust.

Nous avons généré les keystore et truststore nécessaires, configuré Spring Boot pour vérifier les certificats entrants, et testé la communication via *curl* et Postman.

Du côté du frontend, la mise en place directe du mTLS dans Angular n'étant pas possible avec un navigateur, nous avons contourné cette limitation en implémentant un **serveur proxy Node.js** capable de gérer les certificats et d'établir une communication mTLS avec le backend. Ceci montre une compréhension réaliste des contraintes techniques des environnements modernes.

Ce projet a également permis d'aborder des notions essentielles telles que les algorithmes cryptographiques recommandés aujourd'hui, les risques identifiés dans l'OWASP concernant les certificats et la gestion des clés, ainsi que l'importance d'une chaîne de confiance bien construite.

En conclusion, ce travail nous a donné une vision complète de l'implémentation sécurisée d'un système client-serveur via SSL/TLS et mTLS, depuis la théorie cryptographique jusqu'à l'intégration pratique dans un workflow logiciel réel. Il illustre à la fois l'importance de la sécurité dans les applications modernes et la nécessité de maîtriser les outils (OpenSSL, Keytool, cURL, Spring Security, Node.js) permettant de la mettre en œuvre efficacement.